

Deployment Plan for the Notification Platform

Overview

This deployment plan is based on the design and requirements in the Notification Platform Prototype document. The platform must be capable of sending more than 50 million highly personalized emails per week, handle both blast and triggered campaigns, extract user-specific data from a relational database, merge it with dynamic templates, deliver mail via SMTP, and track events (opens, clicks, bounces, unsubscribes). The architecture calls for a micro-services approach to meet scalability, reliability and maintainability requirements. The plan below maps those requirements into a cloud-native deployment with containerized services, managed databases, message queues, CI/CD pipelines, monitoring and scaling.

Architecture Summary

Micro-services and roles

The document proposes seven core micro-services:

Service	Role
Controller Service	Orchestrates campaigns, exposes API for triggered email events.
Harvester Service	Uses a work queue and a multi-threaded model to extract personalized data for each recipient from the relational database. Processes users in batches (~200 users at a time) to avoid large result sets and uses a read replica for high-throughput queries.

Distributor Service	Takes harvested data, merges it with the appropriate template and creates an SMTP envelope for each recipient. Uses a template engine (e.g., Jinja2) that supports variables and control flow; Jinja templates contain variables and tags that control logic (e.g., loops and conditionals) and allow conditional expressions (e.g., {% if loop.index is divisibleby 3 %}).
Sender Service	Delivers SMTP envelopes to one or more SMTP servers. Implements domain-based throttling to avoid deliverability problems; throttling addresses cases where an ISP limits the number of messages accepted from a sender . It maintains connection pools and implements retry logic with domain-specific limits.
Bounce Processor	Processes bounce notifications (hard/soft/spam) and maintains suppression lists.
Tracker Service	Generates tracking pixels and click-through URLs, records opens/clicks/bounces in a separate tracking database.
Admin Service	Provides UI/API for campaign management, template editing, segmentation queries and viewing metrics.

Supporting components

- **Relational Databases:** The primary database (PostgreSQL) holds user data and campaign configuration. A read-replica is used by the Harvester to offload read traffic. A

separate tracking database stores sends, opens, clicks and bounces to isolate analytics workloads.

- Queue: A database-backed queue coordinates work between services; however, for scalability it is recommended to use a message queue service (e.g., Amazon SQS or RabbitMQ).
- Configuration/Secrets: Service configuration should not be hard-coded. External configuration management (e.g., AWS Secrets Manager, HashiCorp Vault) is used to store database credentials, SMTP credentials and domain throttling limits. The micro-services deployment guide notes that a centralized configuration system is essential for handling environment-specific settings .
- Messaging: Event-driven communication between services (e.g., Harvester → Distributor → Sender) can be implemented via message queues. The micro-services article highlights that many architectures use asynchronous messaging for loosely coupled services .
- Service Discovery / API Gateway: For dynamic service discovery, a registry (e.g., AWS Cloud Map, Consul) and an API gateway should be used. An API gateway acts as the single entry point, handling authentication, rate-limiting and routing .
- Observability: Aggregated logging, metrics and tracing are crucial for diagnosing distributed systems . Tools such as the ELK stack or AWS CloudWatch for logs, Prometheus/Grafana for metrics and OpenTelemetry for traces should be integrated.

Deployment platform

The plan targets AWS ECS (Fargate) or Kubernetes (EKS) as the container orchestration platform. Micro-services are packaged as containers, consistent with best practices for micro-services deployment . Orchestration handles container placement, horizontal scaling and rolling updates. An external load balancer and an API gateway provide ingress, and service discovery resolves internal services.

Infrastructure Setup

1. Networking and VPC

1. Virtual Private Cloud (VPC): Create a VPC with public and private subnets across at least two availability zones for high availability.

2. Security Groups: Define security groups to restrict inbound/outbound traffic. Allow inbound HTTP/HTTPS from the internet to the load balancer and restrict inter-service traffic to only necessary ports (e.g., database, queue, SMTP).
3. Load Balancer and API Gateway: Provision an Application Load Balancer (ALB) or API Gateway in the public subnets. Route incoming traffic to the Controller and Admin services. Use TLS certificates for HTTPS.

2. Databases and Storage

1. Primary Database: Deploy an Amazon RDS PostgreSQL instance in private subnets. Create a read replica for the Harvester to run micro-queries concurrently. Enable automatic backups and point-in-time recovery.
2. Tracking Database: Provision a separate PostgreSQL instance to store sends, opens, clicks and bounces. This separation prevents heavy analytics queries from slowing down transactional operations.
3. Object Storage / CDN: Host static assets (e.g., images used in templates) on Amazon S3 with CloudFront distribution for efficient delivery.

3. Queues and Messaging

1. Work Queue: Use Amazon SQS or RabbitMQ to implement the work queue between services. Each Harvester worker reads a batch of recipients from the queue, processes it and pushes the result to a distribution queue for the Distributor. If using a database-backed queue initially, design tables to store job status and attempt counts, but plan to transition to SQS for high-volume campaigns.
2. Event Bus: Use AWS SNS or EventBridge for system-wide events (e.g., bounce notifications, unsubscribes) to trigger downstream actions (e.g., bounce processor).
3. Rate-limiting & Throttling: Maintain a configuration table or secrets with domain-specific rate limits (e.g., Gmail: 100/hr; Yahoo: 50/hr). The Sender service enforces these limits; adjust dynamically based on ISP feedback. Throttling is necessary because ISPs may limit how many messages they accept from a sender .

4. Container Orchestration

1. ECS/EKS Cluster: Create an ECS cluster with Fargate tasks or an EKS cluster. Define tasks for each micro-service with appropriate CPU/memory allocations. Use service

definitions to keep a desired number of tasks running and enable autoscaling based on queue depth or CPU usage.

2. **Service Discovery:** Configure AWS Cloud Map or a similar service to register each container instance. Use internal DNS to resolve services (e.g., harvester.service.local). Micro-services find each other via discovery instead of hard-coded endpoints .
3. **Deployment Strategy:** Implement blue/green or canary deployments for each service. Blue/green uses two identical environments and switches traffic after validation; it enables safe, zero-downtime rollouts . Canary gradually shifts a small percentage of traffic to new versions, allowing observation of behaviour .
4. **Sidecars and service mesh:** Use sidecar containers (e.g., Envoy) for logging/tracing injection, TLS termination and policy enforcement if adopting a service mesh. Otherwise, integrate logging/tracing libraries in each service.

5. SMTP Infrastructure

1. **SMTP Servers:** Use a cloud-hosted SMTP service (e.g., Amazon SES, SendGrid) with dedicated IPs. Configure multiple outbound connections to handle throughput (10+ connections). Authenticate via SPF, DKIM and DMARC records to protect sender reputation .
2. **Domain-based sending:** Send mail from sub-domains dedicated to campaigns (e.g., m.example.com) to isolate deliverability. Warm up new IPs gradually and monitor bounce/complaint rates. provides guidance that sending a large volume of emails from a new account can trigger spam filters.

Service Implementation

Controller Service

- **API Framework:** Use a lightweight framework like FastAPI (Python) to expose REST endpoints for creating campaigns, launching blasts and handling triggered events. Use authentication (JWT or OAuth2).
- **Orchestration Logic:** Validate input, create records in the campaign database, segment the recipient list based on user-provided SQL conditions and enqueue jobs for the Harvester. Maintain campaign status and progress.

Harvester Service

- Batch Processing: Use Celery or a similar distributed task queue; each worker fetches a batch of ~200 recipients, executes micro-queries to gather per-recipient data from user tables and related tables, and stores the result to a staging table or passes it to the Distributor queue.
- Concurrency: Use a thread pool to handle multiple recipients within each task. Use a read replica to avoid contention on the primary database.
- Scaling: Horizontal scaling based on queue length; tasks can be stateless.
- Data Sources: Plan for hybrid data sources (local tables and external APIs) by mapping data requirements per template. Each template includes a JSON definition of required fields.

Distributor Service

- Template Engine: Use Jinja2 for merging data with templates. Jinja templates support variable substitution and control structures—`{% for item in navigation %}` iterates over data , and `{% if loop.index is divisibleby 3 %}` demonstrates conditional logic .
- Message Assembly: For each recipient, load the relevant template from the database, render subject and body with the personalized context, and produce an SMTP envelope (To, Subject, Body, Headers).
- Queueing: Push the completed envelopes to the Sender queue for delivery.

Sender Service

- SMTP Delivery: Maintain a pool of SMTP connections. Use asynchronous sending to maximize throughput. Implement domain-based throttling—limit the rate of sends per domain to avoid ISP throttling .
- Retry Logic: Differentiate between transient errors (mailbox full, temporary rejection) and permanent errors (invalid address) and requeue messages accordingly.
- Monitoring: Log successes and failures; update send status in the database for reporting.

Bounce Processor

- Inbound Processing: Subscribe to bounce notifications from the SMTP service (e.g., SES SNS topics) or poll a mailbox. Parse bounce types (hard/soft/spam) and update suppression lists. Hard bounces should mark the address as permanently suppressed; soft bounces should trigger retries with backoff.

Tracker Service

- Pixel Tracking: Serve a tiny image (1×1 pixel) with a unique tracking ID. Recording a request counts as an open.
- Click Tracking: Generate redirect URLs that log the click and then redirect to the final destination. Use HTTP 302 responses for redirection.
- Data Storage: Write events to the tracking database with timestamps, user IDs, campaign IDs and event types. Use these for reporting and dynamic throttling adjustments.

Admin Service

- Front-end Framework: Use a modern web framework (React, Vue, etc.) to build dashboards for campaign management, segmentation, template editing and metrics. Consume API endpoints from Controller and Tracker.
- Authentication: Implement RBAC with roles such as admin, marketing, and viewer. Store user accounts in the database with hashed passwords or integrate with an external identity provider (Okta, AWS Cognito).

CI/CD and Deployment Pipeline

1. Source Repositories: Maintain each micro-service in its own Git repository or use a mono-repo with clear folder structure. Track infrastructure code (Terraform), Dockerfiles and CI pipelines in version control. Treat infrastructure as code .
2. Continuous Integration:
 - Run unit and integration tests for each service on every commit.
 - Perform static code analysis and security scanning of Docker images.
 - Build container images using multi-stage Dockerfiles to optimize size.

- Push images to Amazon Elastic Container Registry (ECR).
3. Continuous Deployment:
- Use GitLab CI/CD or GitHub Actions to orchestrate deployments. Each service should have its own pipeline that performs tests, builds the image, deploys to a test environment and, upon approval, deploys to staging and production.
 - Automate deployments with blue/green or canary strategies via ECS services or Kubernetes deployments .
 - Parameterize pipeline templates to reuse logic across services .
 - Run integration tests after deployment. Use the provided scripts/integration-test.sh from the document to verify that the API, health endpoints and metrics work.
 - Run performance tests (e.g., Locust) in staging to measure throughput and concurrency.
4. Promotion and Manual Gates: Use pipeline approvals for moving from staging to production. Perform manual checks of deliverability metrics and bounce rates before full rollout.
5. Cleanup Jobs: Provide scripts to tear down dev/demo environments when no longer needed. The document's cleanup.sh scripts demonstrate removing resources and cleaning data.

Monitoring and Observability

1. Centralized Logging: Collect logs from all containers using a log driver (e.g., AWS FireLens with Fluent Bit) and send them to CloudWatch or Elasticsearch. Include trace IDs and campaign IDs in logs to correlate events across services.
2. Metrics: Use Prometheus or CloudWatch metrics to track queue lengths, throughput (emails/hour), success/failure rates, bounce rates, latency of template rendering and DB query times. Set alarms on thresholds to trigger alerts.
3. Distributed Tracing: Implement tracing (OpenTelemetry, AWS X-Ray) to follow a request from API through Harvester, Distributor and Sender . This helps identify bottlenecks and debug issues in a distributed system.

4. Dashboard: Provide Grafana or CloudWatch dashboards summarizing key metrics. Show progress of large campaigns, error rates, and domain throttling status.

Security and Compliance

- Authentication & Authorization: Secure all API endpoints with OAuth2/JWT. Use API Gateway or service mesh to enforce authentication.
- Data Encryption: Encrypt data at rest (RDS storage, SQS/SNS messages) and in transit (TLS for HTTP and SMTP).
- Secrets Management: Store secrets in AWS Secrets Manager or Vault. Inject them into containers at runtime; do not commit secrets to source control.
- Access Control: Use IAM roles with least privilege to access AWS services. Use network segmentation (private subnets, security groups) to protect databases.
- Compliance: Implement GDPR/CCPA compliance: maintain suppression lists and honor unsubscribes across campaigns.
- Audit Logging: Log administrative actions (campaign creation, template edits) with user IDs and timestamps for traceability.

Scaling & Performance Considerations

- Batch Size & Parallelism: The Harvester should process small batches (~200 users) to avoid long SQL queries and reduce memory usage. Use concurrency at two levels: multiple Harvester workers and multi-threading within each worker.
- Autoscaling: Configure ECS/EKS services to autoscale based on queue length and CPU/memory usage. The Sender service should scale according to the number of pending envelopes.
- Read Replica Usage: Perform all heavy reads on the replica to prevent contention on the primary database. Write operations (sends tracking, campaign status) go to the primary database.
- Domain-based Throttling: Continuously adjust per-domain throttling based on bounce and complaint feedback. Use real-time metrics from the Tracker to identify deliverability issues early.

- **Testing & Simulation:** Before launching a 20 M recipient blast, run load tests that simulate the complete workflow (data extraction → template rendering → SMTP sending → tracking) to ensure the infrastructure can handle the load.

Step-by-Step Deployment Guide

1. Prepare Infrastructure using Terraform:

- Create VPC, subnets, security groups, ALB/API Gateway, ECS/EKS cluster, RDS instances (primary and replica), tracking database, SQS/SNS resources and secret manager entries.
- Apply the infrastructure in dev first, verify that services can connect, then repeat for staging and production.

2. Set up CI/CD pipelines:

- Define pipeline templates for building, testing and deploying each service. Parameterize variables like environment name and AWS region.
- Include steps for docker build, docker push to ECR, terraform apply for infrastructure changes, and ecs update-service or kubectl rollout for deploying new container images.

3. Containerize the services:

- Write Dockerfiles with multi-stage builds. Use a base Python image for dependencies and a slim runtime image for final containers.
- Add health-check endpoints to each service for ECS health monitoring.

4. Implement core functionality:

- Build Controller API endpoints for campaigns and triggered events.
- Implement Harvester, Distributor, Sender, Bounce Processor, Tracker and Admin according to the roles defined above. Ensure that templates use Jinja2 features for loops and conditions .
- Integrate database access via SQLAlchemy or another ORM. Use connection pools and handle transactions properly.

5. Deploy to development environment:

- Run terraform apply for the dev environment.
- Execute the CI/CD pipeline to build and deploy each service to ECS/EKS.
- Seed test data (e.g., 10 k users, sample templates and campaigns) using the provided seeding scripts.
- Verify that health endpoints return OK and that a small blast/triggered send works end-to-end.

6. Run integration and performance tests:

- Use the integration test script to create a campaign, launch it, trigger an event, and verify metrics. Ensure all endpoints (Admin, Controller, Tracker) respond correctly.
- Use Locust or JMeter to simulate load on the API and measure response times and throughput. Adjust container sizes and scaling policies accordingly.

7. Deploy to staging:

- Promote images and configurations to staging via pipeline approval.
- Run Blue/Green deployment: deploy new tasks alongside existing ones, perform smoke tests, then shift traffic. Roll back if tests fail.
- Evaluate domain-based throttling and monitor deliverability metrics. Adjust configuration if necessary.

8. Deploy to production:

- After successful staging, deploy to production with the same Blue/Green strategy.
- Closely monitor metrics (CPU, memory, queue depth, success/failure rates, bounce rates).
- Warm up new sending IPs gradually to maintain sender reputation .

9. Operate and Iterate:

- Continuously monitor logs, metrics and traces. Tune thread pools, batch sizes and throttling parameters.
- Update templates and campaigns via the Admin interface.
- Add new features such as A/B testing, real-time analytics dashboards and machine-learning-based send-time optimization.
- Refactor and introduce additional micro-services as the system evolves. Follow best practices: containerize services, domain-oriented design, shared tooling, automated tests, and CI/CD automation .

Conclusion

This deployment plan translates the requirements in the Notification Platform Prototype into a scalable, maintainable cloud-native architecture. By leveraging container orchestration, queues, a relational database with read replicas, a robust template engine, and a carefully designed CI/CD pipeline, the system can handle very high volumes of personalized email while maintaining deliverability and reliability. Following best practices for micro-services—such as containerization, automated deployments, shared observability tooling and domain-oriented service boundaries —will help ensure that the platform can evolve and scale to meet future demands.